AIT 203: Optimization Project Report

Team Members: Vedansh Patel, Vedant Savani, Tanuj Shah
Roll Numbers: BT2024162, BT2024191, IMT2024018

Question 1: Regression on Bike Sharing Demand

1.1 Introduction

The objective is to predict the demand for bike sharing services using the kaggle dataset. We formulate this as a regression problem, aiming to minimize the Mean Squared Error (MSE) between the predicted and actual count of rentals. We compare three distinct model architectures: Linear Regression, Polynomial Regression, and a Quadratic Model with interaction terms.

1.2 Data Preprocessing

To ensure robust model performance and prevent data leakage, we implemented the following preprocessing steps:

1.2.1 Feature Extraction

The raw timestamp was decomposed into `Year`, `Month`, and `Day`. To capture the cyclical nature of time (23:00 is close to 00:00), we transformed the `Hour` feature using sine and cosine functions:

$$x_{hr_sin} = \sin\left(\frac{2\pi \cdot hr}{24}\right), \quad x_{hr_cos} = \cos\left(\frac{2\pi \cdot hr}{24}\right) \tag{1}$$

Categorical variables (`Season`, `Weather`) were One Hot Encoded so the model wouldn't assume any ranking between categories, they were encoded using $k-1$ binary variables (3 columns instead of 4). We excluded one reference category to avoid collinearity. If all 4 categories were included, the sum of these binary columns would be equal to the column corresponding to the constant term (ones). This linear dependence would make the matrix singular and impossible to invert.

1.2.2 Train-Test Split and Scaling

We split the dataset into training and testing sets prior to any scaling to ensure no train-test leakage occurs.

- **Scaling:** We scaled the numerical features so that they share a similar range by centering them around the mean and adjusting for variability: $x' = \frac{x-\mu}{\sigma}$. This helps the model treat all features fairly rather than giving more importance to those with larger numeric values (Years,Days).
- **Leakage Prevention:** The mean (μ) and standard deviation (σ) were computed only on the training set. These parameters were then applied to transform the test set. This ensures the model never sees test data statistics during training.

1.3 Methodology

1.3.1 Mathematical Formulation

For all models, we employed the analytical solution to the Least Squares problem (Normal Equation). Given a data matrix X (basis) and target vector $\mathbf{y}$, the optimal weights $\mathbf{w}^*$ are computed as:

$$\mathbf{w}^* = (X^T X)^{-1} X^T \mathbf{y} \tag{2}$$

In our implementation, we used the Moore Penrose pseudoinverse $(X^T X)$ (`np.linalg.pinv`) to handle singular basis matrix.

1.3.2 Model Architectures

We evaluated three variations of basis functions $\phi(\mathbf{x})$ (n = 17):

1. **Linear Regression (Baseline):** Uses the raw features.

$$\phi(\mathbf{x}) = [1, x_1, x_2, \ldots, x_n]$$

2. **Polynomial Regression (Degrees $d = 2, 3, 4$):** Includes power terms for each feature independently, excluding interaction terms.

$$\phi(\mathbf{x}) = [1, x_1, \ldots, x_n, x_1^2, \ldots, x_n^2, \ldots, x_1^d, \ldots, x_n^d]$$

3. **Coupling Quadratic Regression:** Includes degree 2 terms and all pairwise interactions $(x_i x_j)$ to capture correlations between features (temperature + working day).

$$\phi(\mathbf{x}) = [1, x_1, \ldots, x_n, x_1^2, \ldots, x_n^2, x_1 x_2, x_1 x_3, \ldots, x_{n-1} x_n]$$

1.4 Results and Metrics

The models were evaluated on the Test Set using Mean Squared Error (MSE) and the Coefficient of Determination (R^2).

Table 1: Performance Metrics on Test Data

Model	**MSE**	$\mathbf{R^2}$
1. Linear Regression	15688	0.5165
2. Coupling Quadratic	10944	0.6627
3. Polynomial Regression ($d = 2$)	13658	0.5790
4. Polynomial Regression ($d = 3$)	12357	0.6191
5. Polynomial Regression ($d = 4$)	12139	0.6259

1.5 Interpretation

1.5.1 Best Performing Model

Based on the results above, the Quadratic model (with interaction terms) performed best, achieving the highest R^2 of 0.6627 and lowest MSE.

1.5.2 Bias Variance Tradeoff

- **Linear Model (High Bias, Low Variance):** The linear model likely underfits the data. Bike demand has non linear relationships with features like temperature (demand rises with temp but drops if too hot) and hour (peaks at commute times). A hyperplane is too simple to capture these curves. Even with infinite points a straight line could never capture a parabola.

- **Coupling Quadratic (Balanced):** This model allows for curvature and interaction. The interaction terms are physically significant; for example, the effect of "Hour" on demand depends heavily on whether it is a "Working Day" or "Holiday." Capturing these conditional dependencies significantly improves R^2.

- **Higher Degree Polynomials (High Varianc, Low Bias):** While increasing degree d reduces bias, models with $d = 3$ or $d = 4$ risk overfitting (High Variance i.e. it passes through all training points, but unstable on new data). Without scaling, the weights can grow large to fit noise in the training set, degrading test set performance compared to the simpler quadratic model.

1.5.3 Curvature Effect

Refers to the between features and the target variable. A simple linear model ($y = wx + b$) assumes that as a feature increases, demand must increase or decrease. However real data behaves differently:

- **Temperature:** Demand rises as temperature increases from cold to warm, but falls if it becomes extremely hot. This creates a parabolic relationship that a straight line cannot fit.

- **Time of Day:** Demand is low at night, peaks during morning, drops mid day, and peaks again in the evening. This is a form of high-degree curvature.

  The Polynomial models successfully capture these effects because the squared terms allow the regression line to bend. This is why the polynomial regression is better than linear.

1.6 Citations

1. Scikit-learn. Preprocessing data: https://scikit-learn.org/stable/modules/preprocessing.html#standardization-or-mean-removal-and-variance-scaling

2. Stack Exchange cyclical features: https://datascience.stackexchange.com/questions/5990/what-is-a-good-way-to-transform-cyclic-ordinal-attributes

Question 2 Option A: Constrained Economic Dispatch using Classical Lagrangian Relaxation

1 Problem Statement and Modelling

We consider a one-hour economic dispatch problem for a utility that operates 14 generating units. The goal is to decide the power output of each unit so as to meet a fixed demand at minimum cost, while respecting generator limits, an emission cap, and a minimum renewable energy share.

The 14 generating units are:

- Units 1–5: Coal-based thermal generators
- Units 6–8: Natural gas generators
- Units 9–10: Hydroelectric generators
- Units 11–12: Solar generators
- Units 13–14: Wind generators

Units 11–14 are treated as renewable generators.

The decision variable is

$$x_i \geq 0, \quad i = 1, \ldots, 14,$$

where x_i is the power output of generator i in MW during the hour.

2 Data and Modelling Assumptions

Basic assumptions

- All costs are modelled as quadratic functions:

$$f(x) = \sum_{i=1}^{14} \big(d_i x_i^2 + c_i x_i\big), \quad d_i > 0.$$

- Emissions are linear in power output with fixed coefficients e_i. Hydro, solar and wind units are assumed to have zero emissions.
- Each unit has a minimum/maximum limit $0 \leq x_i \leq \text{cap}_i$.
- Total demand is fixed at $D = 400$ MW.
- The maximum allowed emission is $E_{\max} = 220$ (arbitrary units).
- A minimum of 25% of demand must be met by renewables:

$$x_{11} + x_{12} + x_{13} + x_{14} \geq \alpha D, \quad \alpha = 0.25.$$

- All numerical values for c_i, d_i, cap_i and e_i have been chosen by us to be reasonable and to make the optimization problem well-conditioned and convex.

Data used in the code

The complete data used in the Python code is listed in Table 1. Indices shown in the table are 1-based; the code uses 0-based indexing.

Unit i	Type	d_i	c_i	cap_i (MW)	e_i
1	Coal	0.0040	15	80	0.90
2	Coal	0.0045	16	80	0.95
3	Coal	0.0050	14	75	0.92
4	Coal	0.0042	15	70	0.90
5	Coal	0.0048	17	70	0.93
6	Gas	0.0030	20	60	0.50
7	Gas	0.0032	21	60	0.45
8	Gas	0.0031	19	50	0.48
9	Hydro	0.0020	5	40	0.00
10	Hydro	0.0022	6	40	0.00
11	Solar	0.0010	1	30	0.00
12	Solar	0.0010	1	30	0.00
13	Wind	0.0010	2	25	0.00
14	Wind	0.0010	2	25	0.00

Table 1: Cost, capacity and emission data used in the implementation.

With this data, the dispatch problem is:

$$\begin{aligned}
\min_{x\in\mathbb{R}^{14}} \quad & f(x) = \sum_{i=1}^{14} \left(d_i x_i^2 + c_i x_i\right) \\
\text{s.t.} \quad & \sum_{i=1}^{14} x_i = D = 400, \\
& 0 \le x_i \le \text{cap}_i, \quad i = 1, \dots, 14, \\
& \sum_{i=1}^{14} e_i x_i \le E_{\max} = 220, \\
& x_{11} + x_{12} + x_{13} + x_{14} \ge \alpha D = 100.
\end{aligned}$$

Because $d_i > 0$ and all constraints are linear, this is a strictly convex quadratic program with a unique global optimum.

3 Solution Methods Implemented in Code

The code is organised into two parts:

- Part A: Unconstrained quadratic optimization (only the power balance is enforced by eliminating one variable) solved with Newton's method.
- Part B: Fully constrained problem solved using classical Lagrangian relaxation (LR) with box constraints kept explicitly.

We briefly describe each approach and then focus on LR, which is the final method used to report the optimal dispatch.

3.1 Part A: Unconstrained Newton Method (for illustration)

To simplify the power balance constraint $\sum_{i=1}^{14} x_i = D$, the code eliminates x_{14}:

$$x_{14} = D - \sum_{i=1}^{13} x_i.$$

Defining $y = (x_1, \dots, x_{13})$, the reduced objective is $F(y) = f(x(y))$ with no explicit constraints. Newton's method is:

$$y_{k+1} = y_k - H^{-1}\nabla F(y_k),$$

where H is the constant Hessian of F. Since F is strictly convex quadratic, Newton converges in two iterations from both starting points used in the code.

However, because we removed all inequality constraints, the solution contains negative generator outputs and an unphysically low (even negative) cost. This confirms that unconstrained optimization is not appropriate for realistic dispatch, and we only keep these results as a numerical check.

3.2 Penalty Method (briefly tested)

A simple quadratic penalty approach was also tried in the code by adding squared constraint violations to the objective. In practice, large penalty parameters led to numerical instability and convergence towards $x \approx 0$, violating demand and all operating constraints. Because of these poor results, the penalty method was discarded and is not used for the final results.

3.3 Part B: Classical Lagrangian Relaxation (used method)

For the final solution we use classical Lagrangian relaxation while keeping the box constraints $0 \le x_i \le \mathrm{cap}_i$ explicitly. We relax the following constraints:

- Power balance: $h(x) = \sum_{i=1}^{14} x_i - D = 0$
- Emission cap: $g_{\mathrm{em}}(x) = \sum_{i=1}^{14} e_i x_i - E_{\max} \le 0$
- Renewable minimum: $g_{\mathrm{ren}}(x) = \alpha D - (x_{11} + x_{12} + x_{13} + x_{14}) \le 0$

Lagrangian and multipliers

We introduce multipliers $\lambda \in \mathbb{R}$ (free) for $h(x)$ and $\mu, \nu \ge 0$ for the two inequalities. The Lagrangian is

$$\begin{aligned} L(x, \lambda, \mu, \nu) &= f(x) + \lambda\, h(x) + \mu\, g_{\mathrm{em}}(x) + \nu\, g_{\mathrm{ren}}(x) \\ &= \sum_{i=1}^{14} \left(d_i x_i^2 + c_i x_i\right) + \lambda \left(\sum_{i=1}^{14} x_i - D\right) + \mu \left(\sum_{i=1}^{14} e_i x_i - E_{\max}\right) + \nu \left(\alpha D - \sum_{i=11}^{14} x_i\right). \end{aligned}$$

For convenience we define a correction term

$$\gamma_i = \begin{cases} -\nu, & i \in \{11, 12, 13, 14\} \\ 0, & \text{otherwise}, \end{cases}$$

so that the coefficient of x_i in L is $c_i + \lambda + \mu e_i + \gamma_i$.

Primal update (per-unit closed form)

For fixed (λ, μ, ν), minimizing L over x subject only to box constraints is separable by unit. For each i we solve

$$\min_{0 \le x_i \le \mathrm{cap}_i} \; d_i x_i^2 + (c_i + \lambda + \mu e_i + \gamma_i) x_i.$$

The unconstrained minimizer is

$$x_i^{\mathrm{unb}} = -\frac{c_i + \lambda + \mu e_i + \gamma_i}{2 d_i},$$

and the box-constrained optimal x_i is obtained by clipping:

$$x_i^{k+1} = \Pi_{[0, \mathrm{cap}_i]} \left(-\frac{c_i + \lambda^k + \mu^k e_i + \gamma_i^k}{2 d_i} \right),$$

where $\Pi_{[a,b]}(z) = \min\{\max\{z, a\}, b\}$. This formula is exactly what is implemented in the function `primal_minimizer_given_m` in the Python code.

Dual update (gradient ascent)

After updating x, the code evaluates

$$h(x), \quad g_{\mathrm{em}}(x), \quad g_{\mathrm{ren}}(x),$$

and performs a simple gradient ascent step on the dual variables:

$$\begin{aligned}
\lambda^{k+1} &= \lambda^k + \alpha_{\text{step}}\, h(x^{k+1}),\\
\mu^{k+1} &= \max\{0,\ \mu^k + \alpha_{\text{step}}\, g_{\text{em}}(x^{k+1})\},\\
\nu^{k+1} &= \max\{0,\ \nu^k + \alpha_{\text{step}}\, g_{\text{ren}}(x^{k+1})\},
\end{aligned}$$

with a constant step size $\alpha_{\text{step}} = 10^{-3}$.

To monitor convergence, the code uses a combined constraint violation norm

$$\|v(x)\| = \sqrt{h(x)^2 + \max(0, g_{\text{em}}(x))^2 + \max(0, g_{\text{ren}}(x))^2}.$$

Iterations are stopped when $\|v(x)\|$ becomes numerically negligible (around 10^{-12} in the run shown) or the maximum number of iterations is reached. Two different initial multiplier triplets are used: $(\lambda, \mu, \nu) = (0, 0, 0)$ and $(1, 0.5, 0.5)$. Because the primal problem is strictly convex and the constraints are linear, this LR scheme converges to the unique globally optimal solution.

4 Numerical Results

Constrained Lagrangian Relaxation

For LR, both multiplier initializations converge to the same feasible dispatch. A typical convergence plot of $\log_{10} \|v(x_k)\|$ versus iteration shows a roughly monotonic decrease until the violation reaches machine precision (around iteration 180).

The final solution (from either start) satisfies all constraints:

- Total cost: $f(x^\star) = 3750.2340$
- Total generation: $\sum_i x_i^\star = 400.0000$ MW
- Emissions: $\sum_i e_i x_i^\star = 190.5000 \leq 220$
- Renewable generation: $x_{11}^\star + x_{12}^\star + x_{13}^\star + x_{14}^\star = 110.0000 \geq 100$
- Constraint violation norm: $\|v(x^\star)\| \approx 5.68 \times 10^{-13}$
- Final multipliers: $(\lambda^\star, \mu^\star, \nu^\star) \approx (-15.5532,\ 0.0,\ 0.0)$

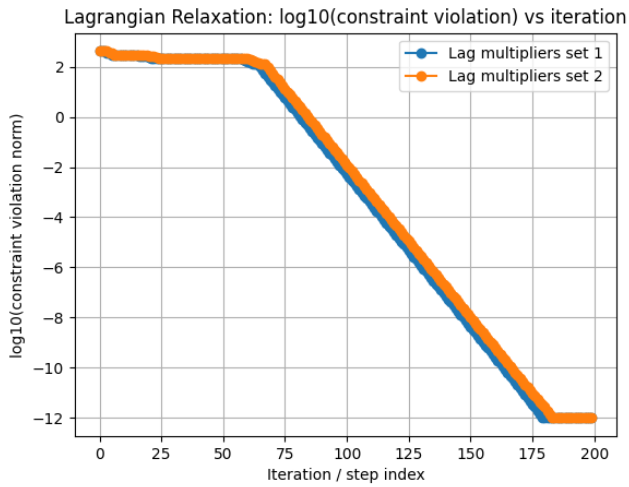

Figure 1: Convergence behavior of the constrained economic dispatch, plotted as $\log_{10}(\|\text{constraint violation}\|)$ vs iteration count.

Final Optimal Dispatch

The optimal generator outputs from the LR algorithm are:

Optimal Generator Outputs (MW)

$$\begin{aligned} x_1 &= 69.1463 & x_8 &= 0.0000 \\ x_2 &= 0.0000 & x_9 &= 40.0000 \\ x_3 &= 75.0000 & x_{10} &= 40.0000 \\ x_4 &= 65.8537 & x_{11} &= 30.0000 \\ x_5 &= 0.0000 & x_{12} &= 30.0000 \\ x_6 &= 0.0000 & x_{13} &= 25.0000 \\ x_7 &= 0.0000 & x_{14} &= 25.0000 \end{aligned}$$

$$\sum_{i=1}^{14} x_i = 400 \text{ MW}$$

Coal units 1, 3 and 4 operate close to their capacities; other coal and all gas units are turned off in the optimal solution. Hydro units run at their full capacity, and all renewable units (solar and wind) are also fully dispatched to meet both the demand and the minimum renewable fraction requirement at the lowest possible cost.

5 Conclusion

In this project, we developed and resolved a 14-unit system's constrained economic dispatch problem. When inequality constraints were disregarded, a straightforward unconstrained Newton method, while mathematically elegant, produced negative generator outputs and an unreasonably low cost. Additionally, a simple penalty method did not produce results that could be used.

A workable, stable, and globally optimal solution was obtained by applying classical Lagrangian relaxation with explicitly enforced generator limits and dual variables handling the remaining constraints. The final dispatch precisely satisfies demand, adheres to the emission cap and the renewable share constraint, and makes the greatest use of inexpensive hydro and renewable energy sources. Overall, the project shows how Lagrangian concepts and convex optimization can be applied practically to power system operation.

References

- A. J. Wood, B. F. Wollenberg and G. B. Sheblé, *Power Generation, Operation, and Control*, Wiley, 2013.
- S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press.
- D. Bertsekas, *Nonlinear Programming*, Athena Scientific.
- Prof. Aswin Kannan, *Class notes*, IIIT-B.